

1. Backend Transaksi & Integritas Data

Bagian ini mengatur seluruh logika bisnis pembelian, pembayaran, dan pencatatan riwayat transaksi secara aman menggunakan **Express.js** dan database yang dikelola oleh `schema.prisma` via **Prisma ORM**.

Mengapa menggunakan Prisma ORM?

1. **Keamanan Data (*Rollback Otomatis*)**: Jika proses transaksi (kurangi stok, simpan pesanan, dll.) ada yang gagal di tengah jalan, seluruh proses dibatalkan secara otomatis agar database tidak korup atau menyimpan data setengah jadi (*Atomic Transactions*).
2. **Kode Lebih Ringkas**: Tidak perlu menulis query SQL manual (BEGIN, COMMIT, ROLLBACK) yang panjang dan rentan kesalahan.
3. **Mencegah Error Ketik (*Type Safety*)**: Fitur *auto-complete* mendeteksi kesalahan nama tabel atau kolom sebelum kode dijalankan di server.
4. **Bebas Ganti Database**: Mempermudah migrasi database (misalnya dari SQLite ke PostgreSQL/Supabase) tanpa menulis ulang kode transaksi Anda.

A. Alur Checkout

- **File Kode**: `checkout.js`
- **Endpoint**: `POST /api/checkout`

Endpoint (Alamat Tujuan): Pintu masuk khusus pembayaran di server backend.

POST (Metode HTTP): Digunakan untuk mengirimkan payload data belanja baru dari frontend ke server.

`/api/checkout` (Jalur/Path): Alamat URL spesifik untuk mengarahkan request ke controller transaksi.

- **Keamanan**: Dilindungi oleh middleware `verifyToken` (hanya user dengan token JWT valid yang bisa checkout).
- **Fungsi Utama (*\$transaction*)**: Menjamin konsistensi data dengan membungkus 5 langkah berikut ke dalam satu transaksi database:
 1. **Validasi & Pengurangan Stok**: Memeriksa stok produk di database. Jika kurang, memicu throw Error (*rollback*); jika cukup, stok langsung dikurangi.
 2. **Pembuatan Pesanan**: Membuat baris data pesanan utama di tabel pesanan dengan status awal PROCESSED.
 3. **Detail Pesanan**: Mencatat rincian barang yang dibeli (jumlah barang, harga satuan, dan subtotal) ke tabel `detail_pesanan`.
 4. **Simulasi Pembayaran**: Menyimpan catatan pembayaran dengan status PAID (Lunas)

sesuai metode bayar yang dipilih ke tabel pembayaran.

5. **Pembersihan Keranjang:** Menghapus seluruh item di tabel keranjang milik user tersebut karena pembelian sudah selesai.

B. Alur Riwayat & Pembatalan

- **File Kode:** transactions.js
- **Mengambil Riwayat (GET /history):** Mengambil semua data pesanan dari user yang sedang login, diurutkan dari yang terbaru (desc). Menggunakan relasi database untuk menyertakan (*include*) data produk dan status pembayaran secara real-time.
- **Membatalkan Transaksi (POST /cancel/:idPesanan):** Membatalkan pesanan secara aman dengan langkah-langkah:
 1. Mengubah kolom status pesanan dan pembayaran di database menjadi CANCELLED.
 2. **Mengembalikan Stok:** Mengambil detail item pesanan, lalu mengembalikan (menambahkan) stok produk tersebut seperti sedia kala.

2. API Entry Point (Gatekeeper Backend)

- **File Kode:** index.js (Backend) File ini merupakan pintu gerbang utama untuk menjalankan server backend Express.js.
- **Inisialisasi Server:** Mengatur konfigurasi variabel lingkungan (dotenv), mengaktifkan cors agar frontend dapat berkomunikasi lintas asal, dan mengaktifkan middleware express.json() untuk membaca body data berformat JSON.
- **Koneksi Database:** Menginisialisasi PrismaClient untuk menjembatani server ke database cloud.
- **Pendaftaran Route (Global Routing):**
 - /api/auth \$→ Autentikasi, Registrasi, dan Login user.
 - /api/products \$→ Manajemen katalog dan pengambilan data produk.
 - /api/cart \$→ Operasi CRUD (tambah/hapus) keranjang belanja user.
 - /api/checkout \$→ Logika inti transaksi pembayaran.
 - /api/transactions \$→ Riwayat transaksi dan pembatalan pesanan.
- **Port Listener:** Menjalankan server agar siap mendengarkan permintaan di Port 5000.

NOTE

Komunikasi Frontend-Backend: Berjalan secara asinkron (*asynchronous*) menggunakan protokol HTTP dengan format pertukaran data JSON. Frontend menggunakan library **Axios** untuk mengirimkan permintaan (*HTTP Request*) beserta token keamanan JWT di bagian header ke backend untuk diolah, lalu backend mengembalikan respon (*HTTP Response*) agar halaman frontend ter-update secara otomatis.

3. Frontend Pages & Alur Koneksi API

Bagian ini menjelaskan bagaimana tampilan halaman (React) terhubung dengan API backend.

A. Halaman Profil Game & Organisasi

- **File Kode:** ff.jsx, hok.jsx, dan About.jsx
- **Fungsi:** sebagai halaman profil informasi (*Landing Page*).
- Halaman ini **tidak** menembak API transaksi ke database. Mereka membaca data statis lokal (seperti divisions.js) untuk menampilkan informasi roster pemain, peringkat tim, win rate, dan jadwal tanding menggunakan layout Bento Grid.

B. Alur Integrasi Transaksi End-to-End

Proses transaksi belanja dinamis di halaman Store.jsx dan History.jsx berjalan dengan alur visual berikut: mermaid graph TD

A[Halaman Store] -->|Add to Cart & Klik Checkout| B(MidtransModal / Simulator)

B -->|Klik Simulasikan Pembayaran Berhasil| C[POST /api/checkout]

C -->|Simpan Pesanan & Kurangi Stok| D[(Database via Prisma)]

D -->|Respon Sukses| E[Redirect ke Halaman /history]

E -->|GET /api/transactions/history| F[Tampilkan Riwayat & Status LUNAS]

1. **Di Halaman Store (Store.jsx):** User memilih produk dan menyimpannya di keranjang belanja (mengirim request ke /api/cart).
2. **Saat Checkout:** Ketika user klik *Proceed to Checkout*, aplikasi memunculkan modal MidtransModal.jsx sebagai simulator pembayaran.
3. **Simulasi Bayar:** Di dalam modal, user memilih metode pembayaran dan mengeklik "**Simulasikan Pembayaran Berhasil**".
4. **Eksekusi Backend:** Tombol tersebut memicu fungsi processCheckout untuk mengirim data belanja dan metode pembayaran ke backend POST /api/checkout dengan menyertakan token JWT di header.

5. **Menampilkan Riwayat (History.jsx):** Begitu respon sukses diterima, halaman dialihkan ke /history untuk mengambil data transaksi terbaru dan menampilkan statusnya yang kini telah berubah menjadi **LUNAS (PAID)**.